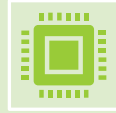


Introduction to GPGPU

GENERAL PURPOSE COMPUTATION ON GPU

Agenda



GPU as a Co-processor



Architectural Support for GPU Programming



GPU Programming Languages

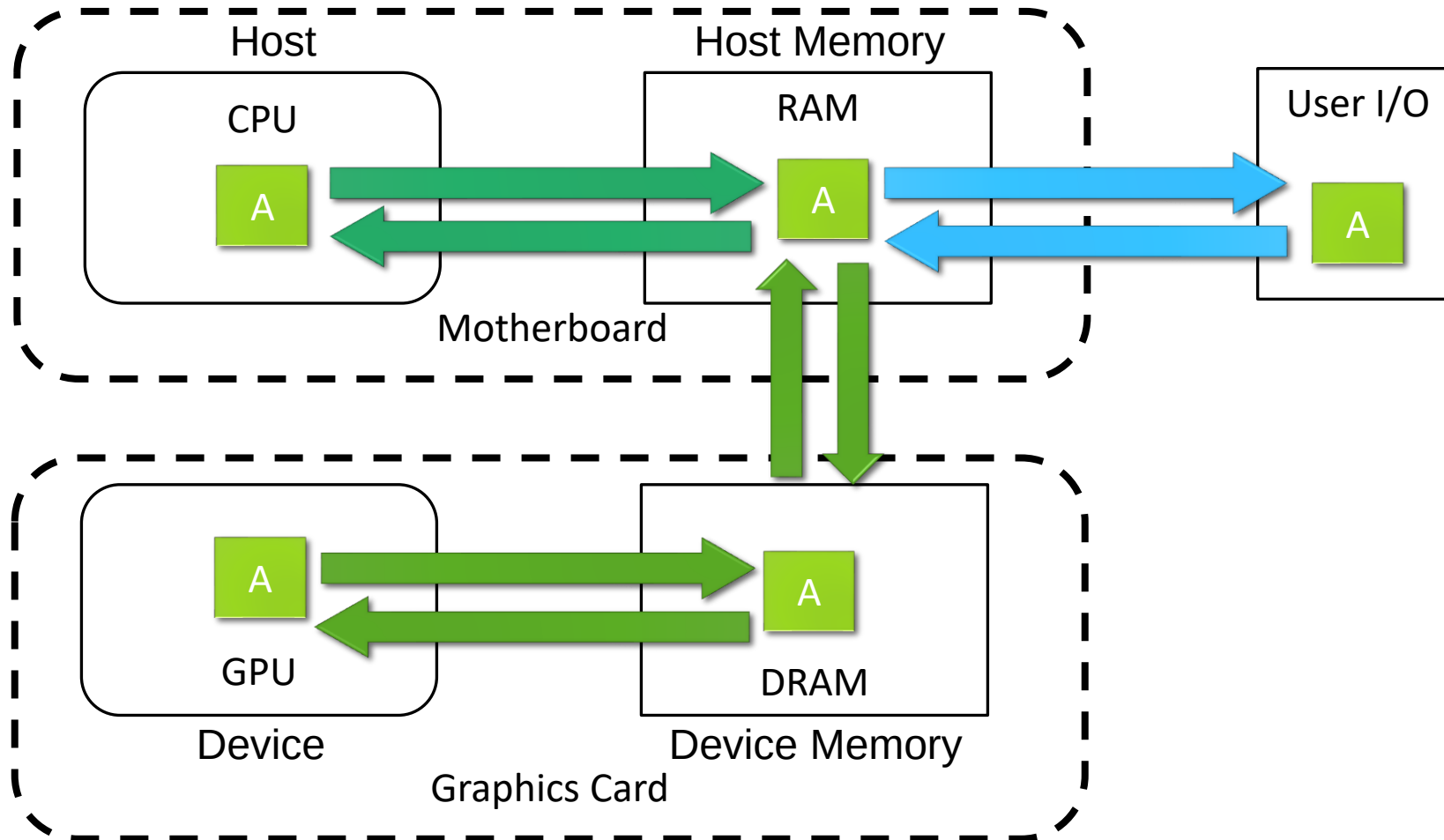


First CUDA Program



CUDA Threading Model

GPU as a Co-processor Sequential Programming Model



GPU Architecture (History)

Problem : How to quickly convert mathematical model of computer graphics into actual signals to be sent to Display Unit?

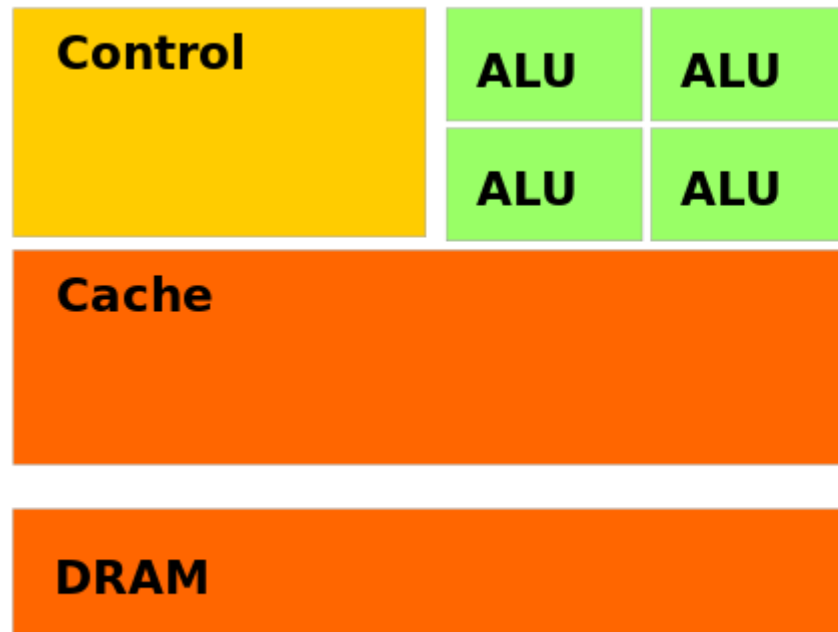
- Break down the graphics processing into a pipeline of dedicated steps & design a hardware for each step
- Graphics Pipeline
 - Vertex Transform & Lighting
 - Triangle Setup & Rasterization
 - Texture & Pixel Shading
 - Depth Test & Blending
 - Framebuffer
- Many stages got added and more and more stages were made programmable
- Ultimately, multiple programmable stages were combined to give an **Unified Scalar Shader Architecture**

Courtesy: David Luebke, SIGGRAPH 2008

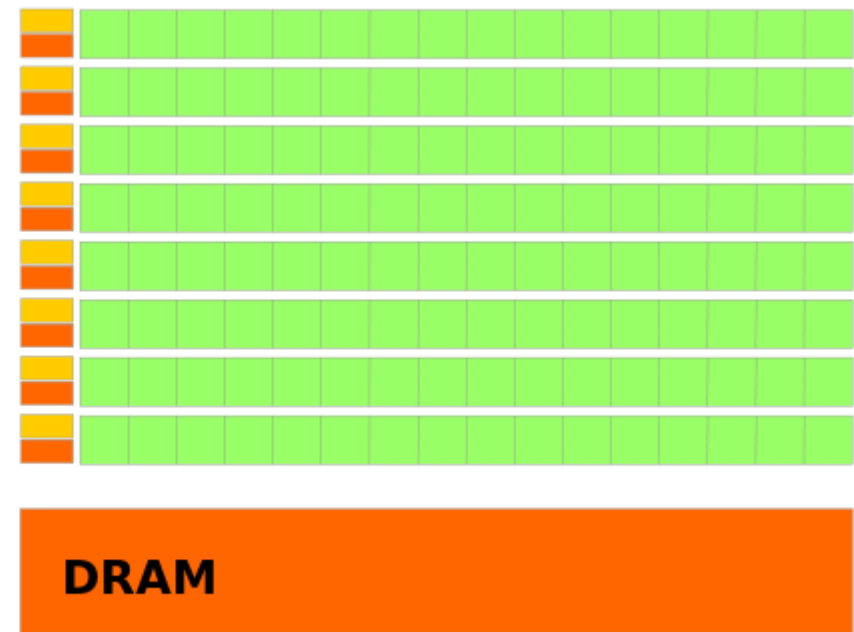
GPU Characteristics

- GPUs were designed for graphics processing where vertices & pixels are processed independently
 - Each processing step is usually arithmetic intensive i.e., multiple operations are applied in between memory access
 - So, less control logic and more of arithmetic logic is required
- GPUs are designed for tasks that can tolerate high latency as long as it can process a lot of tasks in one go (i.e., high latency, high throughput)
 - So, data caching is not a priority
- So, more chip area can be given to ALUs instead of Control Logic and Caches. Therefore, a lot of GPU threads (10s of thousands) can (should) execute at a time.

CPU Vs. GPU: Transistor Allocation Ratio



CPU



GPU

Courtesy: NVIDIA CUDA Programming Guide

Architectural Support for GPU Programming

To use this kind of architecture, following support would be needed:

1. Mechanism to create, schedule and context switch 10s of thousands of threads
2. Mechanism to avoid synchronization issues between so many threads
3. Mechanism to enumerate such large number of ALUs
4. Mechanism to make DRAM available to all threads
5. A simple mechanism to program the user logic and get it executed over GPU hardware

Some Design Choices

1. Thread Management

- Form a threading hierarchy
 - Group of Threads → 1 Block
 - Group of Blocks → 1 Grid
 - 1 Grid → 1 GPU
- Example – to create 10,000 threads in total, one may create:
 - 10 blocks of 1000 threads each
 - 100 blocks of 100 threads each
 - 1000 blocks of 10 threads each

Some Design Choices (Cont..)

2. Thread Synchronization

- Make all threads independent of each other

3. ALU Enumeration

- Form a processor hierarchy
 - Group of Cores → 1 Multiprocessor
 - Group of Multiprocessors → 1 GPU

4. Device RAM Access

- Make whole Device RAM globally accessible to all threads

5. Programming Style

- Follow SIMD, assign same task to all threads on different data

GPGPU Model

Different implementations of GPU architecture takes different design choices but following programming model remains same :

Use GPU as a co-processor with CPU, such that:

- CPU copies the data back and forth into GPU's memory
- CPU executes the memory intensive code
- GPU executes the compute intensive code
- All threads run independent (😊) of each other
- Follows Single Instruction Multiple Data (SIMD) model

This way, GPUs can be used for General Purpose Computations.

Also, this model very well fits into Massively Parallel Programming Model.

Massively Parallel Programming

In MPP, there are

- Large number of processing elements, where
- Each element has to execute a small code (Fine granularity)

Way of parallelization (thinking!) is considerably different than parallelization of regular parallel workload.

GPU Programming Languages

- Kernel Programming Languages
 - Explicit and fine level control over threads and memory operations
 - Example:
 - CUDA for Nvidia GPUs
 - HIP for AMD and Nvidia GPUs
 - OpenCL for AMD and Nvidia GPUs
- Directive-based Programming Languages
 - No control over threads, compiler automatically generates code for parallelization
 - Example:
 - OpenMP for AMD and Nvidia GPUs
 - OpenACC for AMD and Nvidia GPUs

First CUDA Program

Problem-

- Write a program in CUDA to find square of first 500 whole numbers stored in an array.
- Serial implementation-

```
#include <stdio.h>
int main()
{
    int *a, i, N=500;
    a = (int*) malloc (sizeof(int) * N);
    for(i=0; i<N; i++)          a[i] = i;
    for(i=0; i<N; i++)          a[i] = a[i] * a[i];
    for(i=0; i<N; i++)
        printf("Square of %d = %d\n", i, a[i]);
    return 1;
}
```

Parallel Implementation

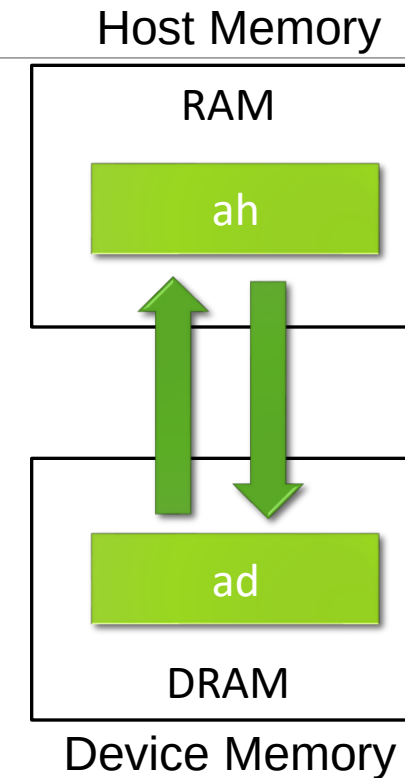
```
#include <stdio.h>
#include <cuda.h>
int main()
{
    int *ad, *ah, i, N=500;

    //allocate memory on host and device
    ah = (int*) malloc (sizeof(int) * N);
    cudaMalloc((void**) &ad, (sizeof(int) * N));

    for(i=0; i<N; i++)          ah[i] = i;

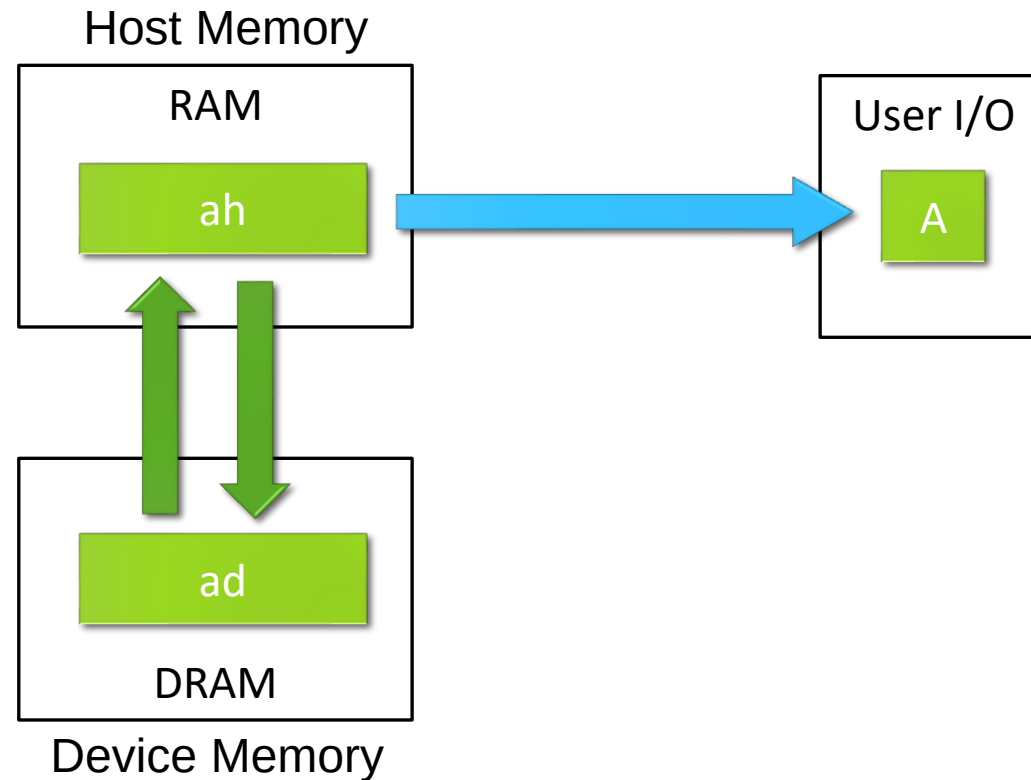
    //copy data from host to device
    cudaMemcpy(ad, ah, sizeof(int) * N, cudaMemcpyHostToDevice);

    //launch CUDA Kernel
    find_square <<< 1, N >>> (ad, N);
```



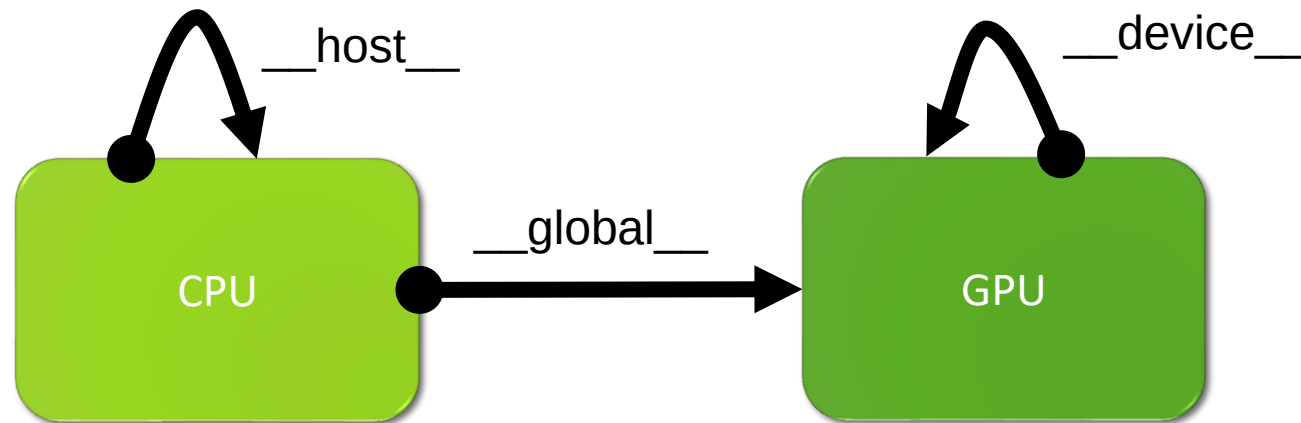
Cont..

```
//copy data from device to host  
cudaMemcpy(ah, ad, sizeof(int) * N, cudaMemcpyDeviceToHost);  
  
for(i=0; i<N; i++)  
    printf("Square of %d = %d\n", i, ah[i]);  
return 1;  
}
```



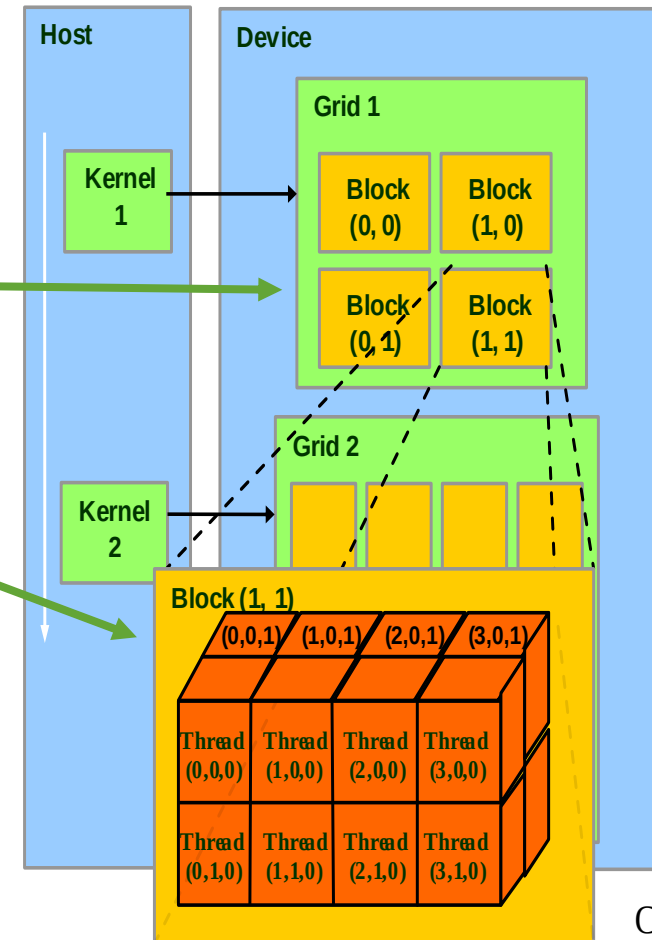
Kernel Code

```
void find_square(int *ad, int N)
{
    int index = threadIdx.x;
    if(index < N)
        ad[index] = ad[index] * ad[index];
}
```



GPU Threading Model

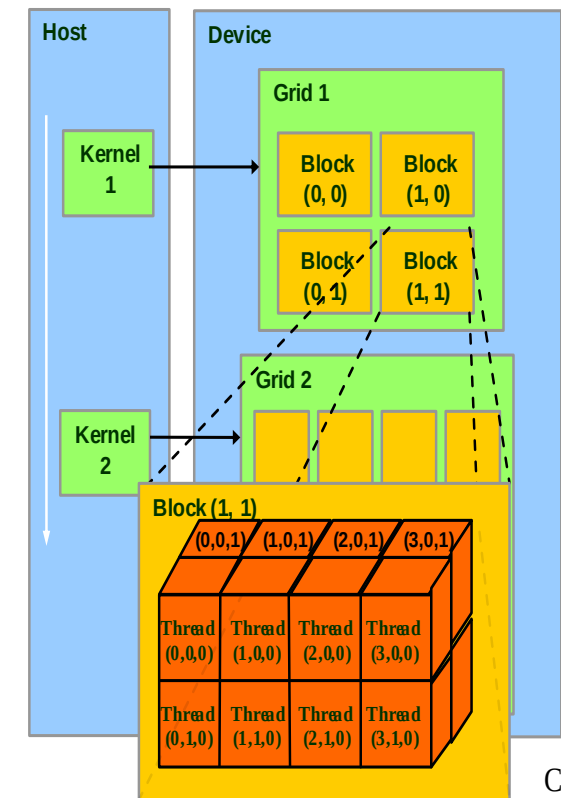
- All threads in a block execute the same kernel program (SPMD)
- Each thread uses IDs to decide what data to work on
 - Block ID: 1D, 2D or 3D
 - Thread ID: 1D, 2D or 3D
- Simplifies memory addressing when processing multidimensional data
 - Image processing
 - Solving PDEs on volumes
- Threads in the same block share data and synchronize while doing their share of the work
- Threads in different blocks cannot cooperate
 - Each block can execute in any order relative to other blocks



Courtesy: NDVIA

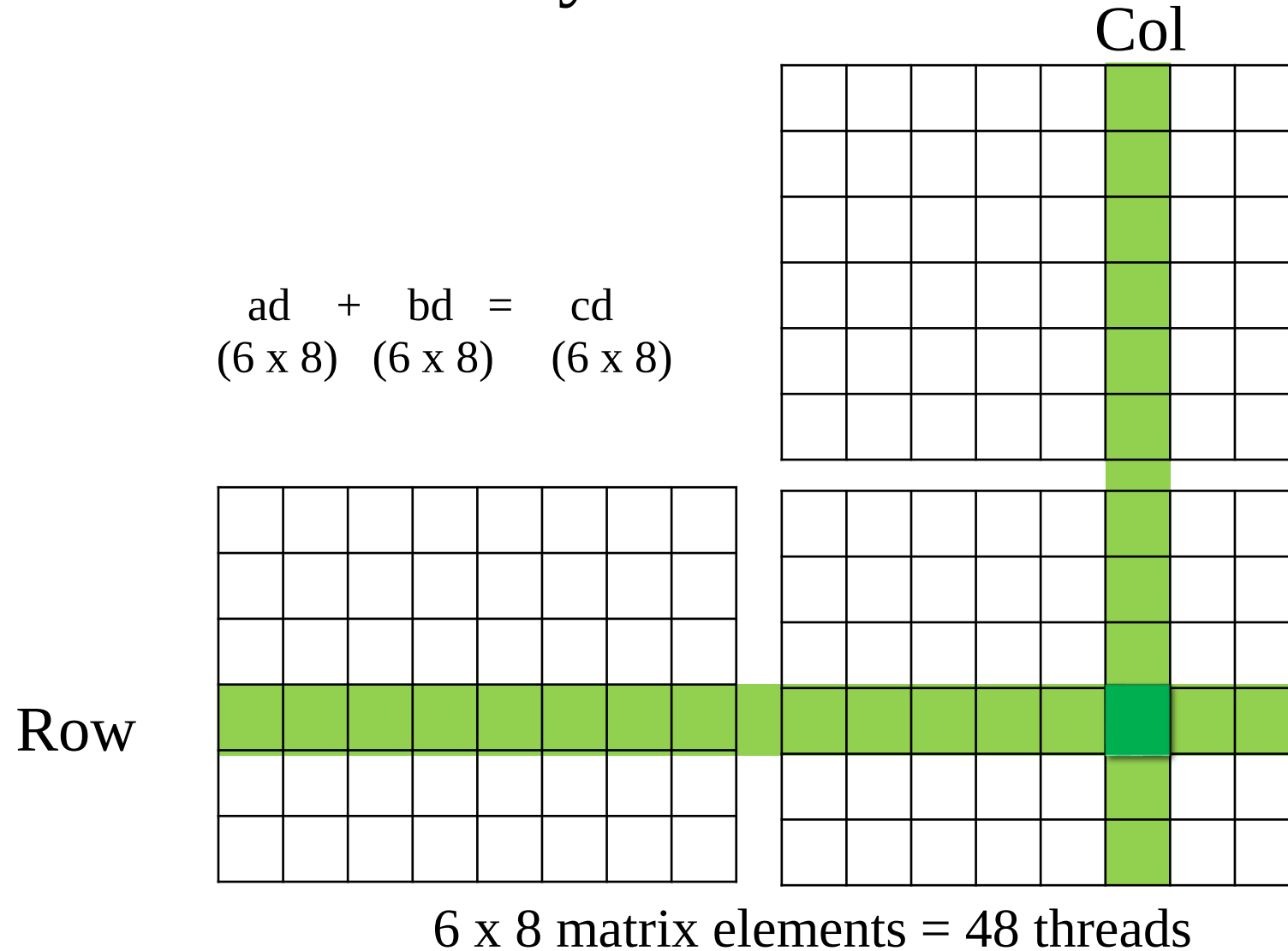
Thread and Block Handles on GPU

- `dim3` struct $\rightarrow$ `uint3` with uninitialized fields set as 1 (max 65536)
- Dimensions of the grid (in terms of blocks) $\rightarrow$ `dim3 gridDim`
 - `gridDim.x`, `gridDim.y`, `gridDim.z`
- Dimensions of the blocks (in terms of threads) $\rightarrow$ `dim3 blockDim`
 - `blockDim.x`, `blockDim.y`, `blockDim.z`
- Block index in the grid $\rightarrow$ `dim3 blockIdx`
 - `blockIdx.x`, `blockIdx.y`, `blockIdx.z`
- Thread index in the block $\rightarrow$ `dim3 threadIdx`
 - `threadIdx.x`, `threadIdx.y`, `threadIdx.z`



Courtesy: NDVIA

Case Study : Matrix Addition



```

__global__ void matrix_add (float *ad, float *bd, float *cd, int N)
{

    int Row = blockIdx.y * blockDim.y + threadIdx.y; //Row means y direction
    int Col  = blockIdx.x * blockDim.x + threadIdx.x; //Col means x direction

    int index = Row * N + Col; //N is the number of elements in a row

    cd[index] = ad[index] + bd[index];

}

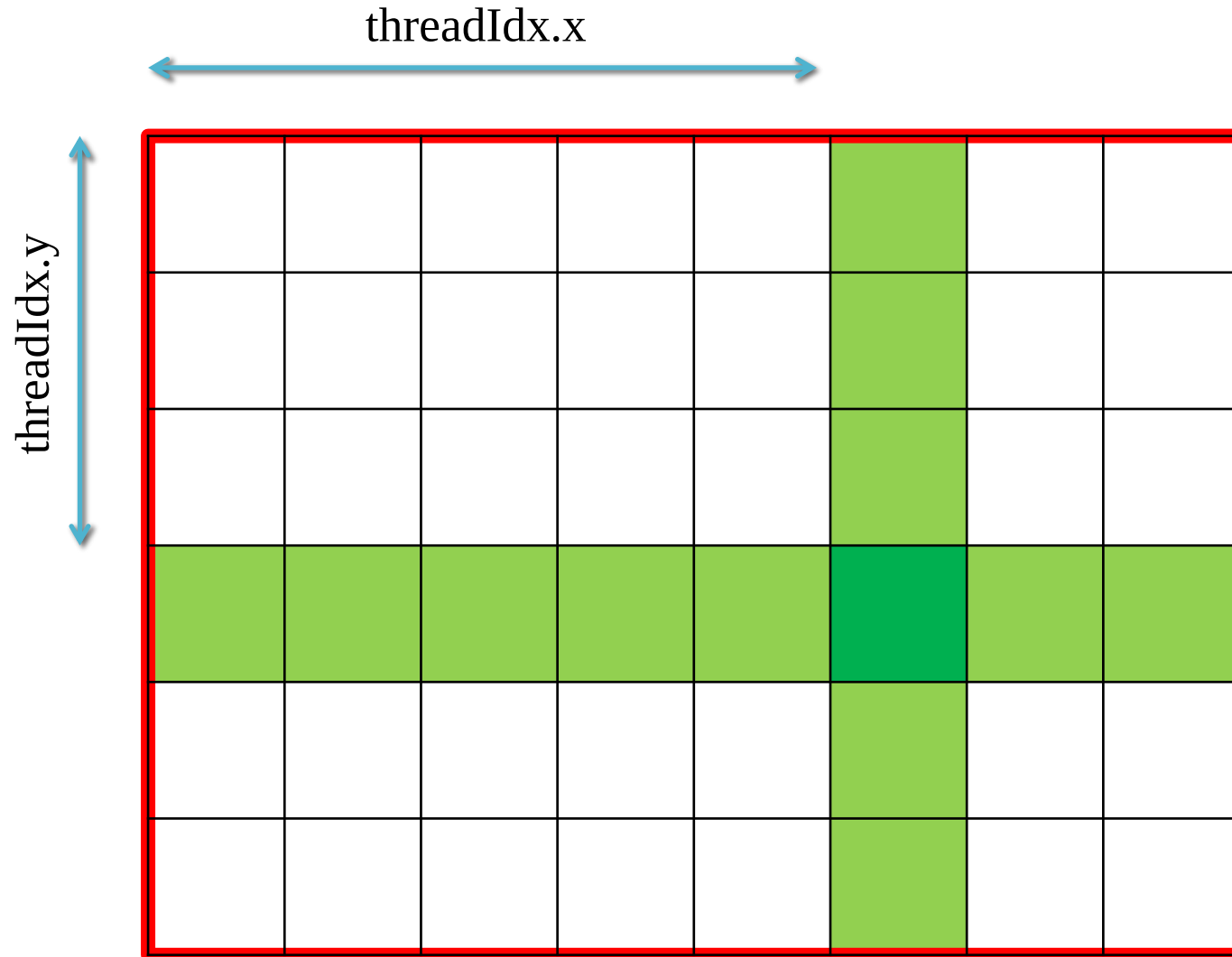
```

```

.
.
dim3 grid_conf( ?, ? );
dim3 block_conf( ?, ?, ? );
matrix_add <<< grid_conf, block_conf>>> (ad, bd, cd, N);
.
.

```

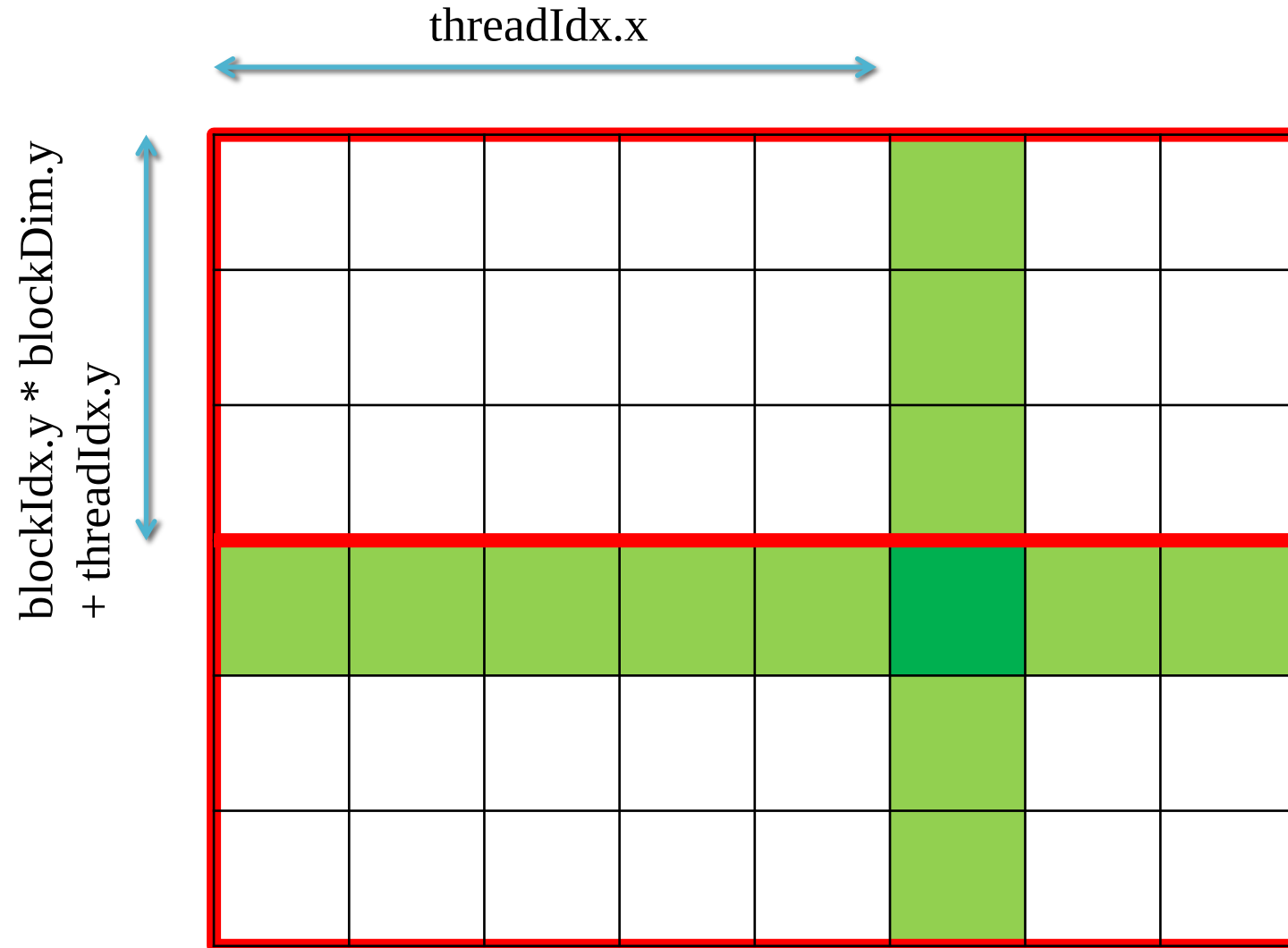
Configuration 1- No of Blocks = 1
No of threads in each Block = (8,6)



Configuration 2-

No of Blocks = 2

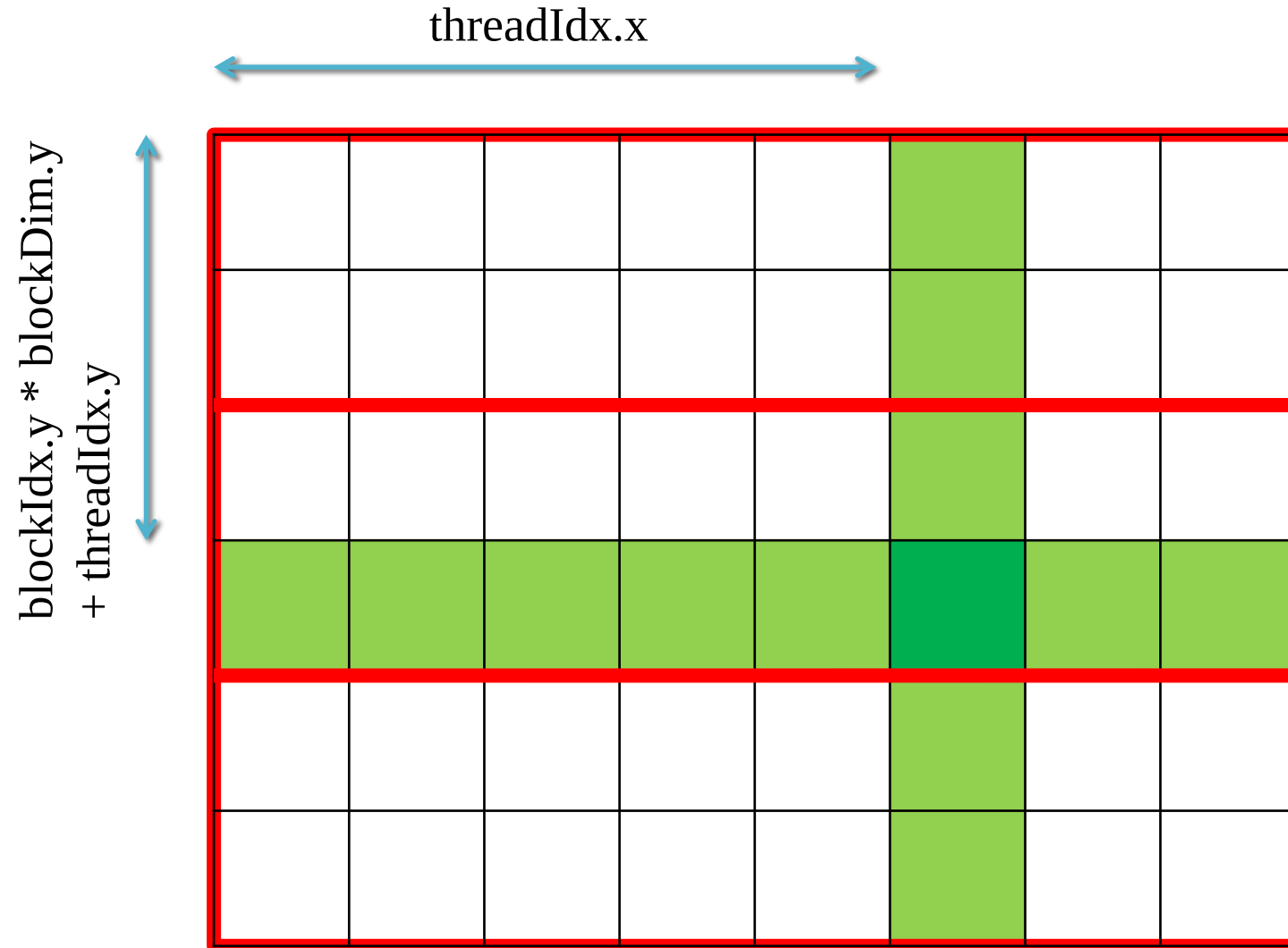
No of threads in each Block = (8,3)



Configuration 3-

No of Blocks = 3

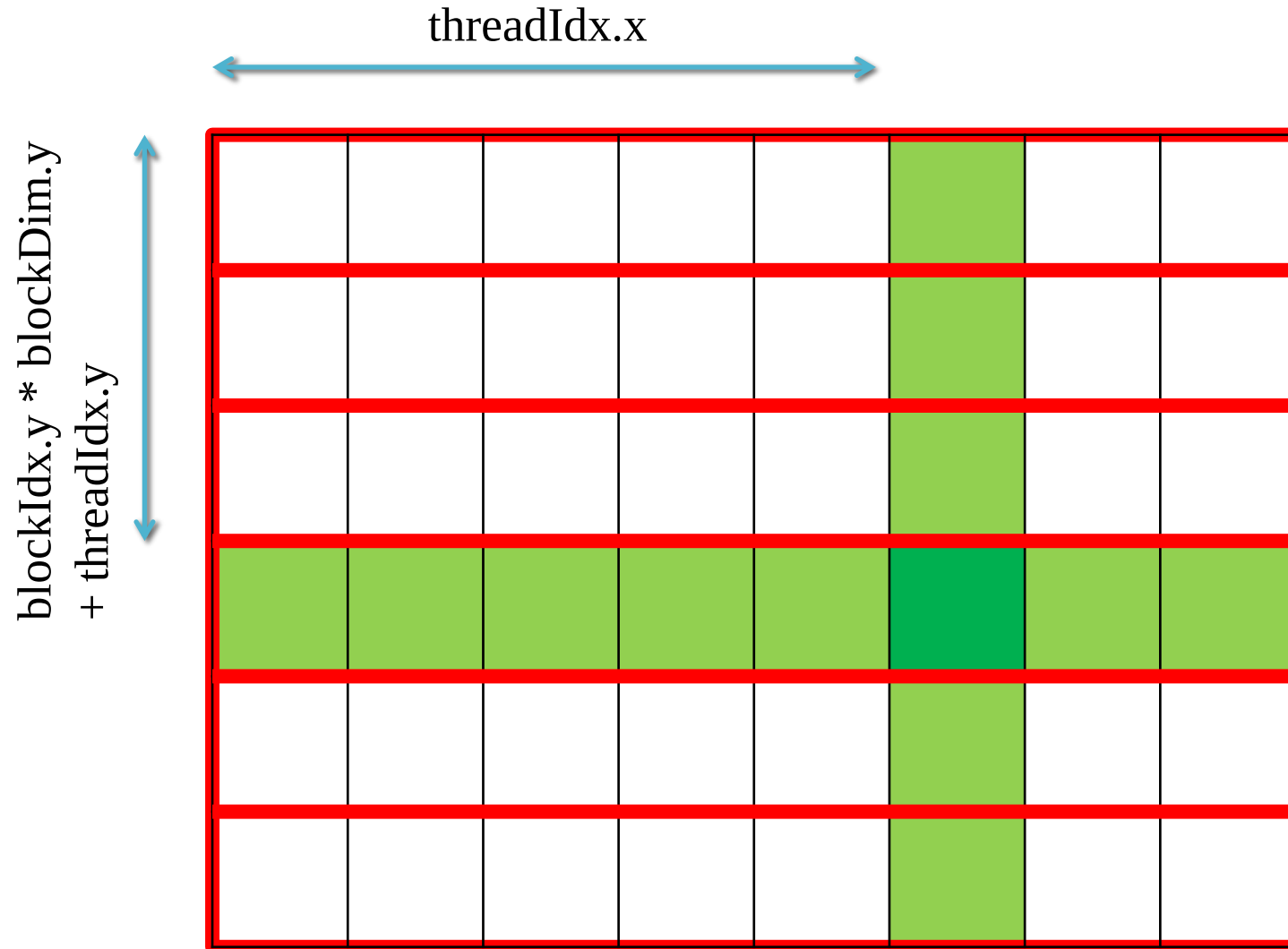
No of threads in each Block = (8,2)



Configuration 4-

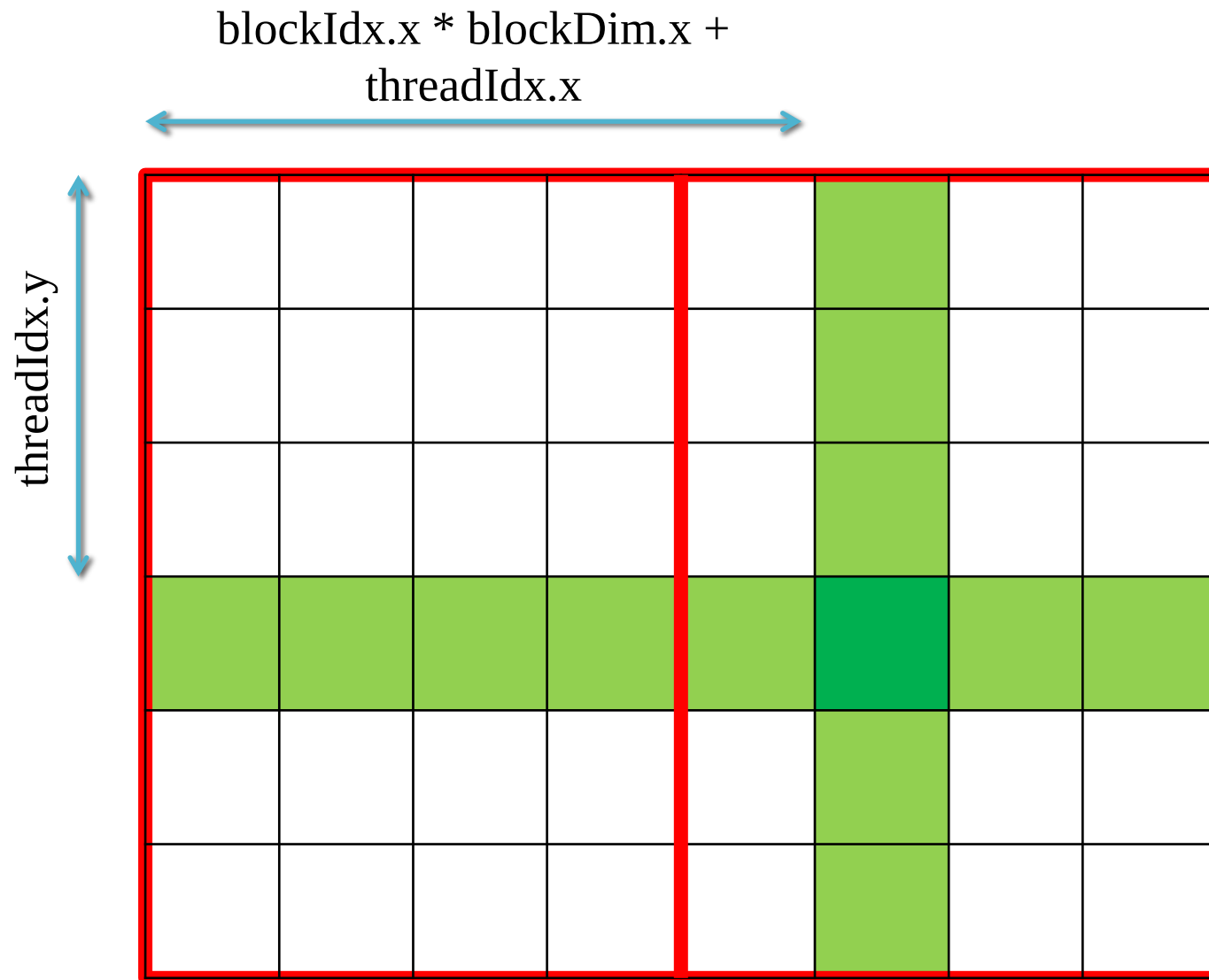
No of Blocks = 6

No of threads in each Block = (8,1)



```
dim3 grid_conf(1, 6, 1);  
dim3 block_conf(8, 1, 1);
```

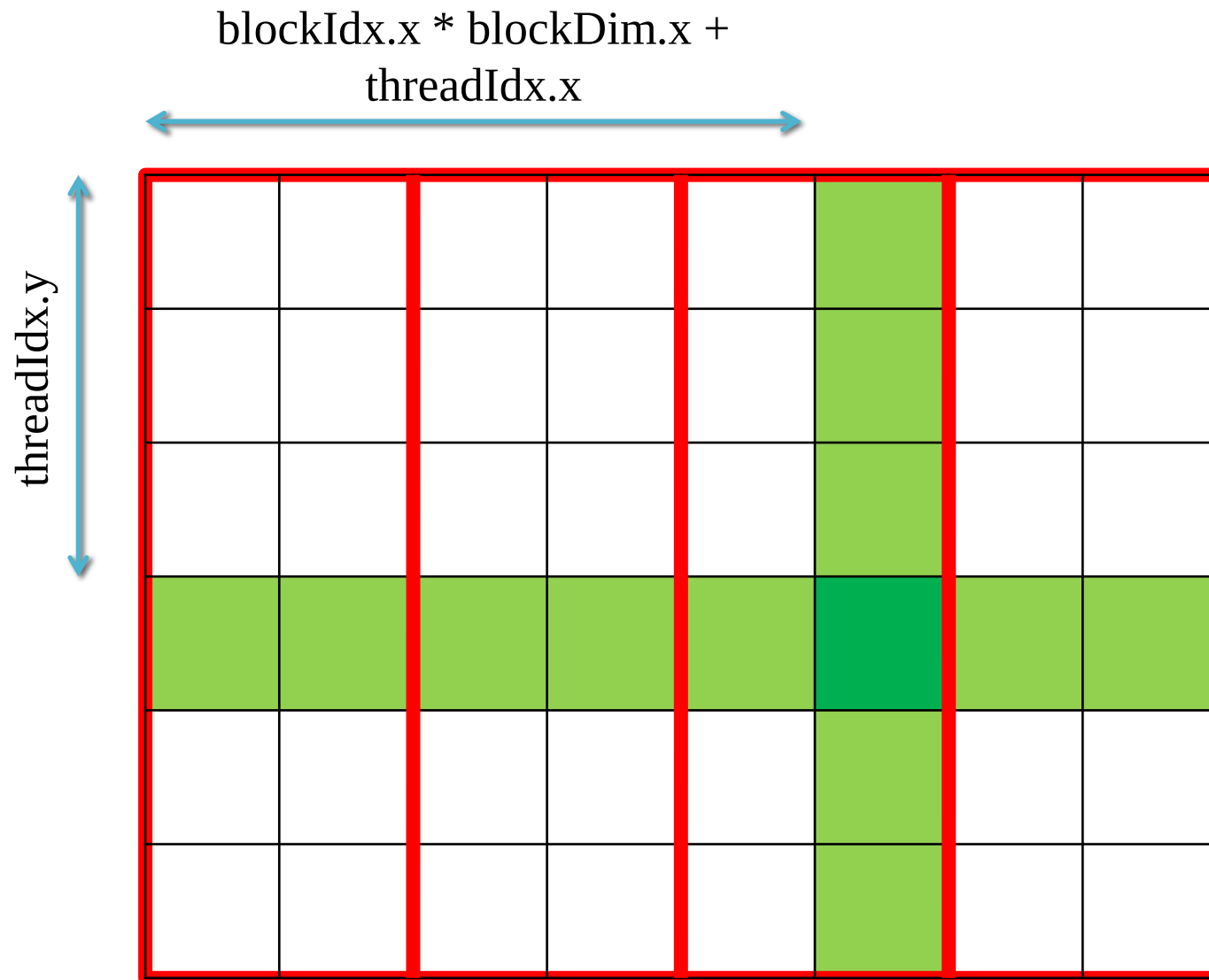

Configuration 5- No of Blocks = 2
 No of threads in each Block = (4,6)



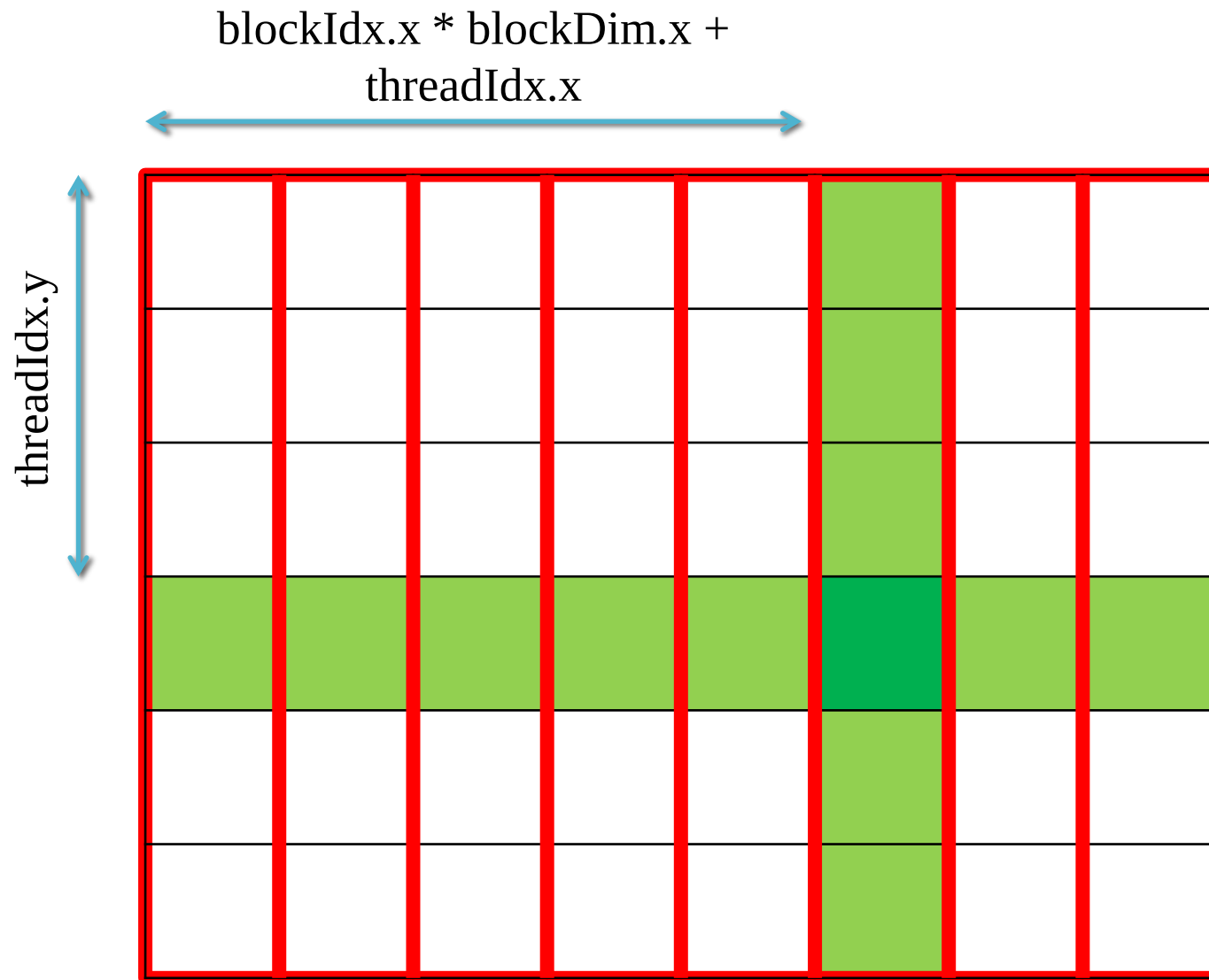
Configuration 6-

No of Blocks = 4

No of threads in each Block = (2,6)



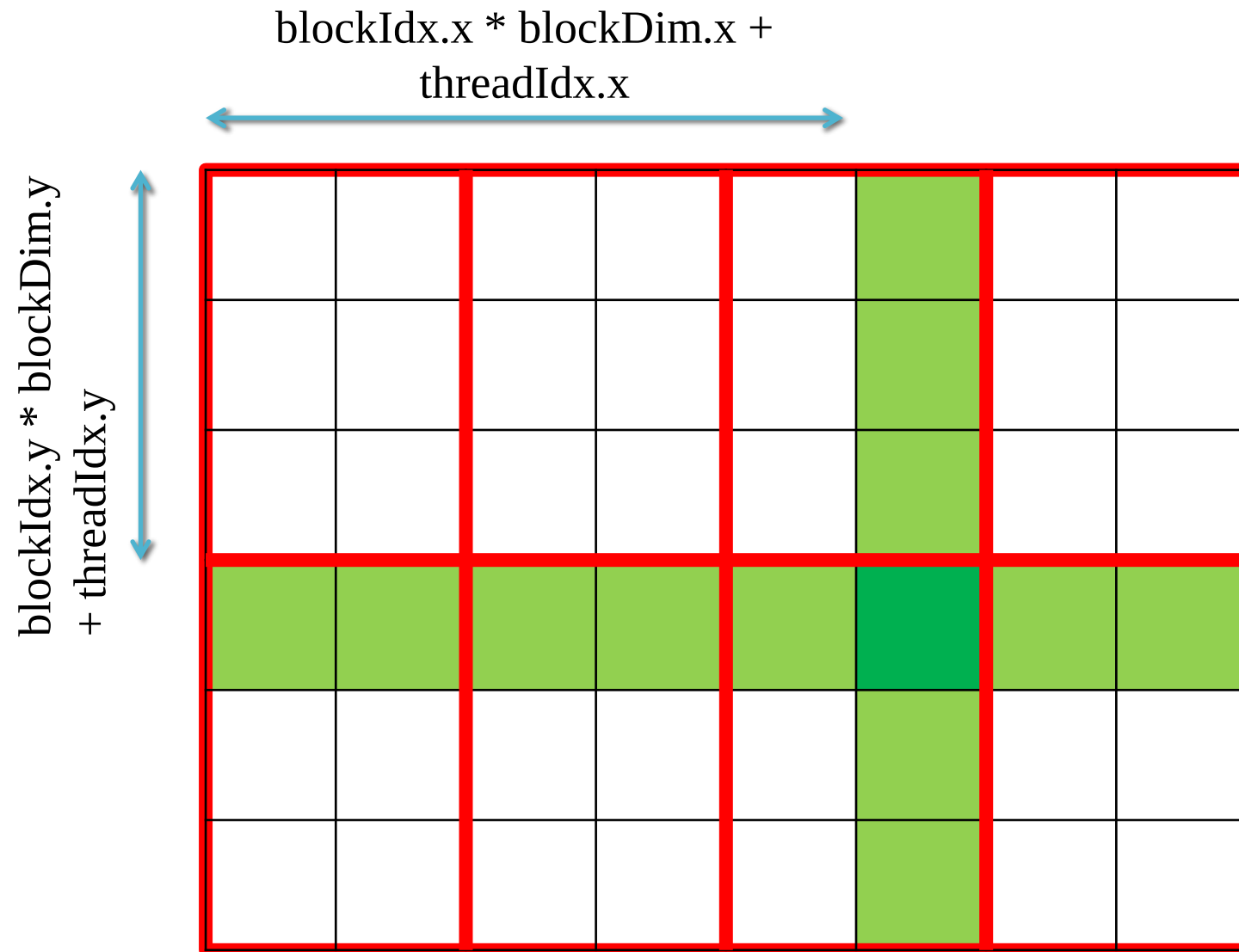
Configuration 7- No of Blocks = 8
 No of threads in each Block = (1,6)



Configuration 8-

No of Blocks = 8

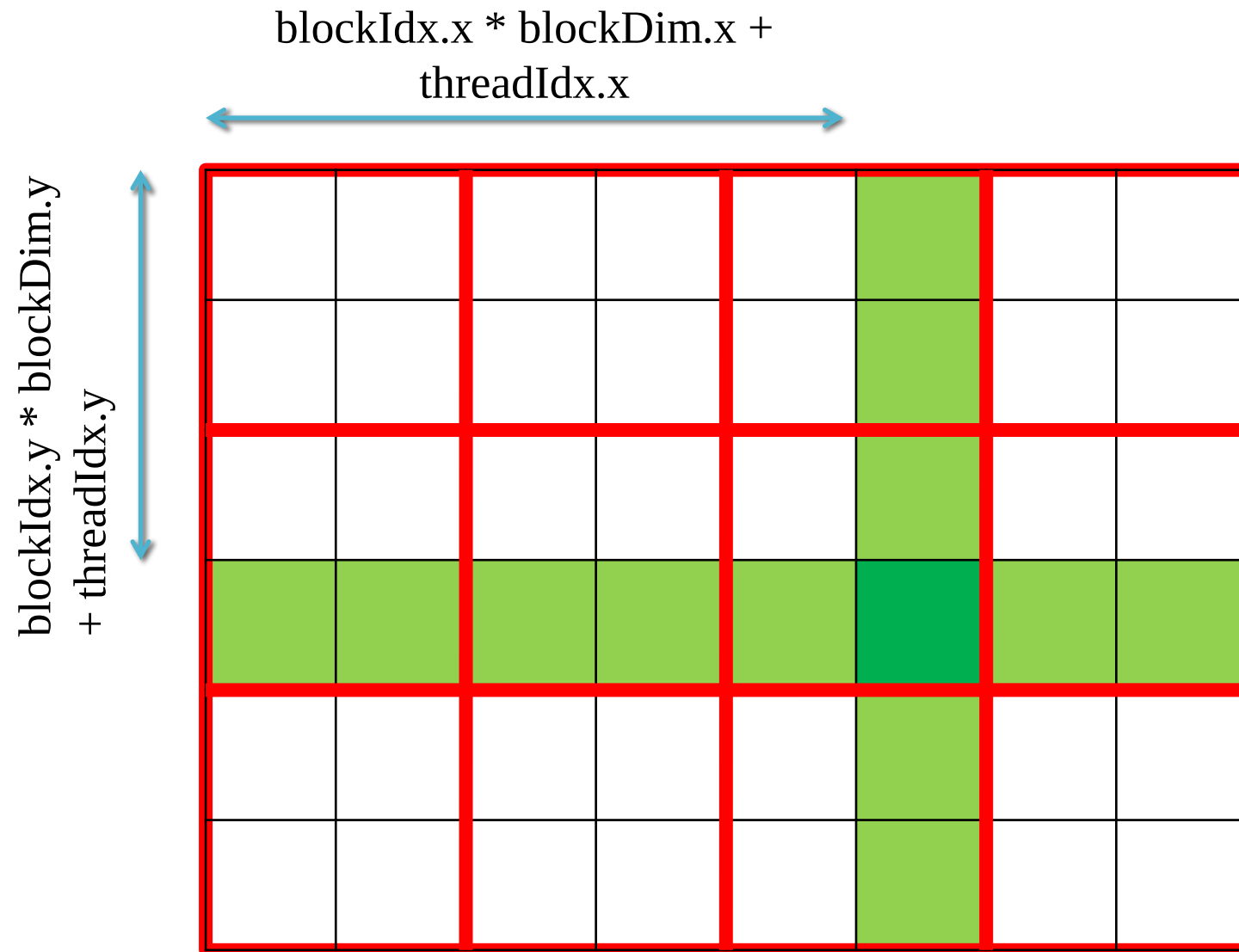
No of threads in each Block = (2,3)



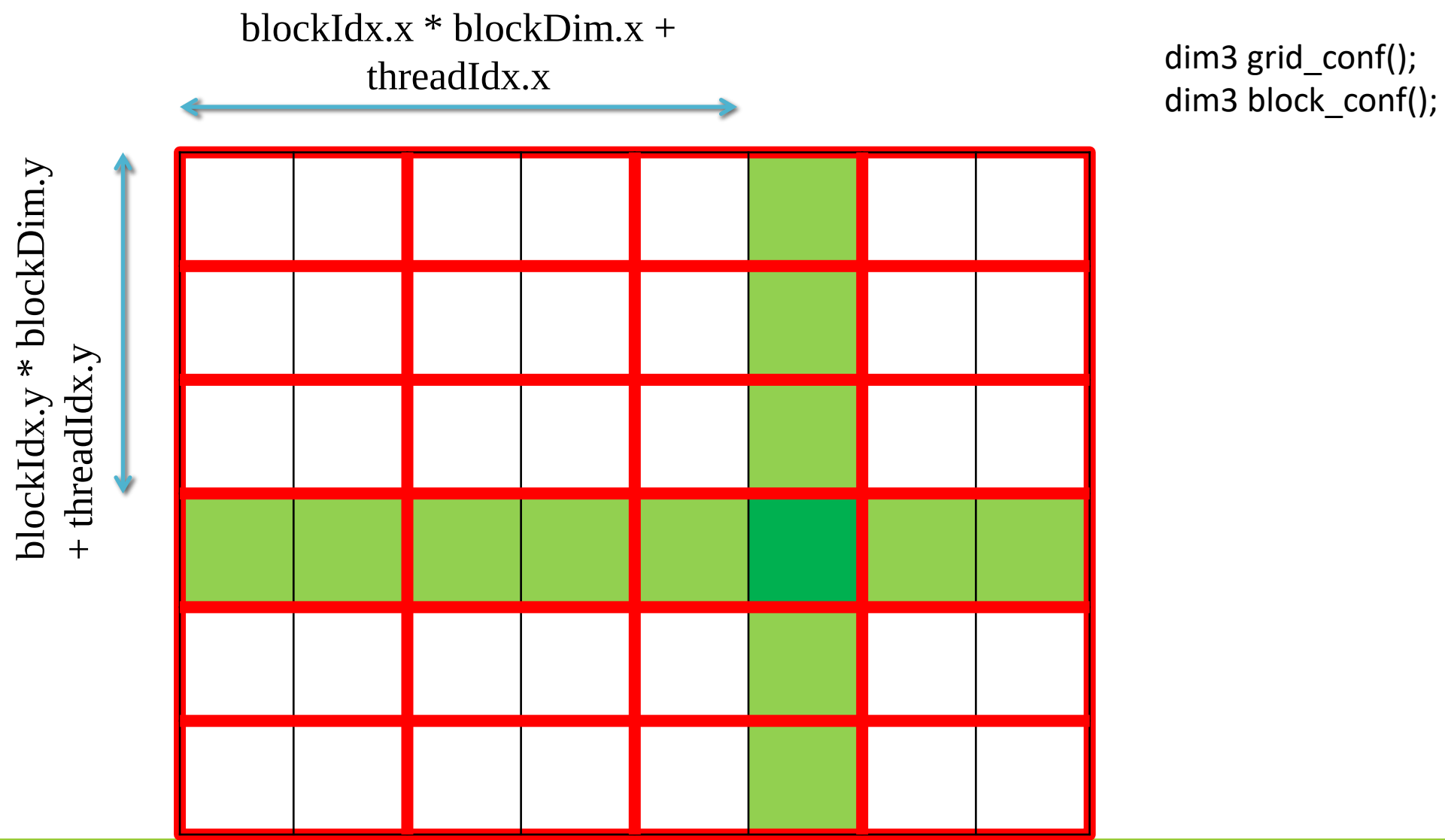
Configuration 9-

No of Blocks = 12

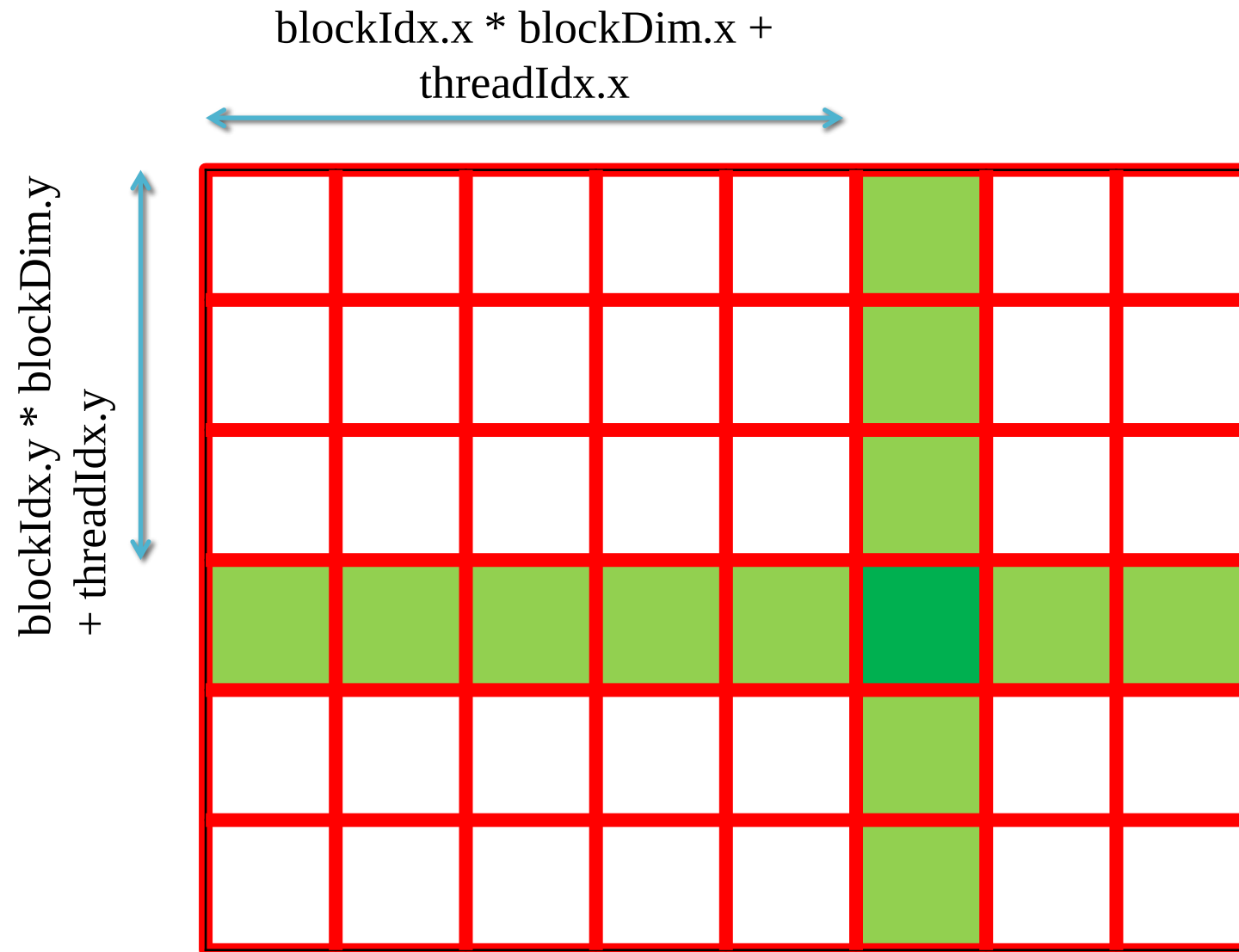
No of threads in each Block = (2,2)



Configuration 10- No of Blocks = 24
No of threads in each Block = (2,1)



Configuration 11- No of Blocks = 48
No of threads in each Block = (1,1)



Questions?
